routing table based on the assigned LIDs.

Installing RDMA

First of all, connect two devices back to back or through a switch. Download and install the latest version of the OFED package from *https://www.openfabrics.org/downloads/*.

OpenFabrics Enterprise Distribution (OFED) is a package developed and released by the OpenFabrics Alliance (OFA), as a joint effort of many companies that are part of the RDMA scene. It contains the latest upstream software packages (both kernel modules and user-space code) to work with RDMA. This package supports most major Linux distributions and CPU architectures.

Extract the *tgz* file and type the following command to start the installation:

```
[root@localhost]# ./install.pl
```

Next, choose '2' (Install OFED software).

From the options displayed, choose '1' (OFED modules and basic user level libraries).

OFED packages will now be installed. Reboot the system to complete the installation.

The structure of a typical RDMA application is as follows:

1. Gets the device list
2. Opens the requested device
3. Queries the device's capabilities
4. Allocates a protection domain
5. Registers a memory region
6. Creates a completion queue
7. Creates a queue pair
8. Brings the queue pair to a ready-to-send state
9. Creates an address vector
10. Posts work requests
11. Polls for completion
12. Cleans up

To identify RDMA-capable devices in your system, type the following command:

```
[root@localhost]# ibstat
```

You need to be aware of the medium you are planning to use for your RDMA connection—InfiniBand or Ethernet. Verify that the ports are *Active* and *Up*.

Getting the device list

ibv_get_device_list( ) returns an array of the RDMA devices currently available.

An example of how this is done is given below:

```
struct ibv_device **dev_list;
dev_list = ibv_get_device_list(NULL);
if (!dev_list)
        exit(1);
```

Opening the requested device

ibv_open_device( ) opens the device and creates a context for further use.

An example is given below:

```
struct ibv_device **device_list;
struct ibv_context *ctx;
ctx = ibv_open_device(device_list[0]);
if (!ctx) {
        fprintf(stderr, "Error, failed to open the device
'%s'\n",
                ibv_get_device_name(device_list[i]));
        return -1;
}
printf("The device '%s' was opened\n", ibv_get_device_
name(ctx->device));
```

Querying the device's capabilities

ibv_query_device( ) returns the attributes of the RDMA device that is associated with a context. These attributes are constant and can be later used.

Here is an example:

```
struct  ibv_device_attr device_attr;
int rc;
rc = ibv_query_device(ctx, &device_attr);
if (rc) {
            fprintf(stderr, "Error, failed to query the
device '%s' attributes\n",
                        ibv_get_device_name(device_list[i]));
            return -1;
}
```

Allocating a protection domain

ibv_alloc_pd( ) allocates a protection domain for an RDMA device context.

An example is given below:

```
struct ibv_context *context;
struct ibv_pd *pd;
pd = ibv_alloc_pd(context);
if (!pd) {
            fprintf(stderr, "Error, ibv_alloc_pd()
failed\n");
            return -1;
}
```

Registering a memory region

ibv_reg_mr( ) registers a memory region associated with the protection domain to allow the RDMA device to perform read/write operations.

Here is an example:

```
struct ibv_pd *pd;
```